# PREDATOR Agent

Praxic Reinforcement and Extinction-Driven
Agentic Task Orchestrator and Realizer

Comprehensive Technical Report
Repository Analysis, Neural Architecture, Training Pipeline,
and Hardware Requirements Assessment

Repository: github.com/Justo-Tapiador/predator-agent
Author of PREDATOR: Justo Tapiador Garcia (Universidad de Alicante)
Report generated: May 2026

# Table of Contents

# 1. Executive Summary

PREDATOR (Praxic Reinforcement and Extinction-Driven Agentic Task Orchestrator and Realizer) is a novel deep agentic AI system built on the Artificial Junky Neuron (AJN) framework, developed by Justo Tapiador Garcia at the Universidad de Alicante (UA). The repository provides a complete reference implementation in JavaScript (Node.js), featuring a 12-layer deep backbone architecture called ANN-Psi, a Hierarchical Command Interpreter (HCI) for translating natural language directives into distributed addiction targets, a Token-Energy Arbitrator (TEA) for resource-efficient output control, and a four-phase training pipeline that progressively shapes neuronal addiction, aligns behavior with owner objectives, and hardens the system against adversarial conditions.

Unlike conventional LLM-based agents that rely on external prompt engineering and global reward functions, PREDATOR operates on the principle of intrinsic motivation. Each computational unit within the network is an autonomous agent (an AJN) that "craves" specific stimuli, explores its environment to satisfy that craving, and self-regulates through a six-phase life cycle comprising Random exploration, Reinforcement, Saturation, Withdrawal, Frustration, and Extinction. This design paradigm is grounded in the Agentic Theory series (WALLERMAX-AI preprints 2604.00012, 2604.00013, 2604.00014), which introduces the mathematical formalism for addiction-driven computation.

The repository is implemented in JavaScript (ES modules), requires Node.js >= 18.0.0, and is licensed under MIT. It includes source code for the core neural architecture, training pipeline, command-line interface, web dashboard, benchmarking tools, and comprehensive test suites. The implementation is currently at version 0.1.0 and carries an explicit warning that the agent must be trained before it can function correctly. The current codebase uses simulated classical transformer blocks and synthetic data generators, indicating that it serves as a research prototype and architectural blueprint rather than a production-ready system.

# 2. Overview: What is PREDATOR?

PREDATOR is best understood as an autonomous AI agent architecture that departs fundamentally from the dominant paradigm of prompt-driven LLM agents. In a standard LLM agent framework, a large language model receives instructions via natural language prompts, generates actions, and receives feedback through external reward signals or human evaluation. The behavior of such systems is externally driven: the agent acts because it is told to act, and it improves because it is rewarded or corrected from the outside. PREDATOR inverts this relationship by making every computational unit intrinsically motivated. Each Artificial Junky Neuron (AJN) within the system has its own internal "craving" level, its own activation threshold, and its own policy for generating output (praxis). The system does not wait for external commands to act; instead, it continuously seeks stimuli that satisfy its internal cravings.

The name PREDATOR is a backronym that captures the core dynamics: Praxic Reinforcement (actions are reinforced through stimulus satisfaction), Extinction-Driven (neurons that fail to find their craved stimuli undergo extinction and reset), Agentic Task Orchestrator (the system orchestrates complex tasks through the collective behavior of its neuronal agents), and Realizer (it actually

executes actions in the environment through its Praxic Stream Executor). The predator metaphor extends to the way individual neurons "hunt" for their preferred stimuli, competing and cooperating within the layered hierarchy.

The repository provides the following key capabilities: (1) A 12-layer deep backbone (ANN-Psi) that processes stimuli through a combination of AJN layers and classical transformer blocks; (2) A Hierarchical Command Interpreter that translates human-readable directives into distributed addiction targets across all network layers; (3) A Token-Energy Arbitrator that dynamically suppresses output when tasks are progressing well (during Saturation phases), achieving claimed token efficiency improvements of 3x-5x over conventional LLM agents; (4) A CascadeMonitor that detects and mitigates extinction cascades where multiple neurons fail simultaneously; and (5) A four-phase training pipeline that progressively shapes the addiction profiles of all neurons from random initialization to adversarially hardened production readiness.

# 3. Theoretical Foundations

The PREDATOR system is grounded in the Agentic Theory series authored by Justo Tapiador Garcia, published as three preprints under the WALLERMAX-AI label. These documents establish the mathematical and conceptual framework that underpins the entire implementation. Understanding these foundations is essential for grasping the design decisions in the codebase.

## 3.1 WALLERMAX-AI 2604.00012: Definition of the AJN

The first preprint defines the Artificial Junky Neuron, the fundamental computational unit. An AJN is formally defined as a five-element tuple AJN = (M, theta, Omega, delta, tau), where M(t) is the craving level in [0,1] representing how much the neuron "wants" its preferred stimulus; theta(t) is the activation threshold in [0,1] that determines when craving triggers action; Omega is the policy generator, parameterized as a Gaussian distribution with mean mu and covariance Sigma that produces praxis vectors; delta is the metabolic decay rate governing how quickly the activation threshold decreases during withdrawal; and tau is the extinction horizon, the number of consecutive failure steps before the neuron resets. The craving update follows an exponential smoothing equation: M(t+1) = beta * M(t) + (1-beta) * alpha(t), where alpha(t) is the stimulus intensity and beta is the smoothing coefficient (default 0.85). This formulation draws an explicit analogy to addiction dynamics in neuroscience, where tolerance (threshold increase), withdrawal (craving return after stimulus removal), and extinction (addiction dissolution after prolonged failure) are well-documented phenomena.

## 3.2 WALLERMAX-AI 2604.00013: The AJN and ANN-Psi

The second preprint extends the AJN definition to multi-layer architectures. It introduces the Agentic Neural Network (ANN-Psi), a deep architecture composed of AJN units organized into three compositional paradigms: Homogeneous (all units share the same stimulus class), Heterogeneous (units are divided into K competing stimulus classes with winner-takes-all selection), and Hybrid (classical neural transformation combined with AJN context modulation). The paper defines the

Deep Agentic Flow (DAF) as the propagation of stimulus tensors through the layer hierarchy, where each layer transforms the stimulus and passes it forward. The concept of lateral inhibition between neurons in the same layer is introduced as a mechanism to prevent collective saturation lock, where all neurons become simultaneously satisfied and output ceases. Cascade risk is defined as the fraction of units approaching extinction in a layer, providing an early warning signal for system-wide collapse.

### 3.3 WALLERMAX-AI 2604.00014: Stimulus Tensor Propagation and TPS

The third preprint details the Tensorial Praxic Stream (TPS), the output mechanism by which the ANN-Psi system emits actions into the environment. The TPS consists of praxis tensors generated by the output layer (Layer 12), which are decoded into structured tool calls by the Praxic Stream Executor (PSE). The paper formalizes the emission rate equation used by the Token-Energy Arbitrator: r_emit(t) = r0 * exp(-kappaE * E_used/E_budget) * (1 + kappaM * M12(t)), where r0 is the baseline emission rate, kappaE is the energy suppression coefficient, and kappaM is the craving boost coefficient. This equation ensures that output is suppressed when energy budgets are depleted and amplified when the output neuron craves more stimulus, creating a natural throttle on token consumption. The paper also describes the CascadeMonitor intervention hierarchy: stimulus injection (mild), task decomposition (moderate), and owner escalation (severe), which progressively address extinction cascades.

## 4. Neural Network Architecture (ANN-Psi)

The ANN-Psi is the central neural architecture of PREDATOR, implementing a 12-layer deep backbone that combines AJN-based agentic layers with classical transformer blocks. The architecture is implemented in src/core/ANNPsi.js and represents a novel departure from standard transformer-based models. Rather than uniform attention layers, ANN-Psi interleaves three distinct layer paradigms with classical blocks, creating a heterogeneous processing pipeline where different levels of abstraction are handled by qualitatively different computational mechanisms.

### 4.1 The 12-Layer Stack

| Layer | Type | Paradigm | Key Configuration | Function |
|---|---|---|---|---|
| 1-2 | Hybrid (Conv+AJN) | Homogeneous | N=64 units alpha=0.3/0.35 | Sensory encoding; perceptual addiction |
| 3 | Heterogeneous AJN | Heterogeneous (K=8) | 8 classes, 8 units each | Low-level feature specialization |
| 4-5 | Transformer | Classical | Attention blocks | Contextual attention |
| 6 | Heterogeneous AJN | Heterogeneous (K=16) | 16 classes, 8 units each | Mid-level concept specialization |

| Layer | Type | Paradigm | Key Configuration | Function |
|---|---|---|---|---|
| 7 | Hybrid (FC+AJN) | Homogeneous | N=128 units alpha=0.5 | Contextual modulation |
| 8-9 | Transformer | Classical | Attention blocks | High-level reasoning |
| 10 | Heterogeneous AJN | Heterogeneous (K=32) | 32 classes, 4 units each | High-order addiction layer |
| 11 | Hybrid (FC+AJN) | Homogeneous | N=256 units alpha=0.6 | Praxic assembly |
| 12 | Output AJN | Homogeneous (N=1) | 1 unit thetaSat=0.80 | TPS Emitter |

*Table 1: The 12-layer ANN-Psi architecture stack.*

The architecture follows a clear abstraction hierarchy. Layers 1-2 handle raw sensory input, blending classical convolution-like feature extraction with AJN-driven perceptual addiction. Layer 3 introduces heterogeneous specialization with 8 competing stimulus classes (syntax, semantics, structure, novelty, relevance, coherence, completion, correctness), enabling the network to develop specialized pathways for different types of input features. Layers 4-5 provide classical self-attention for contextual integration. Layer 6 escalates to 16 competing stimulus classes covering task-level concepts (task_progress, context_depth, tool_success, information_gain, etc.). Layers 8-9 perform high-level reasoning through attention. Layer 10, the deepest AJN layer, uses 32 high-order addiction classes that compute composite stimulus signals from lower-level features. Layer 11 assembles the praxic output, and Layer 12 emits the final Tensorial Praxic Stream as a single output unit.

## 4.2 Forward Pass Data Flow

The forward pass through ANN-Psi proceeds sequentially from Layer 1 to Layer 12. At each AJN layer, the stimulus is processed by all AJN units, which update their internal craving levels, activation thresholds, and praxis policies. The layer aggregates individual unit praxes into a collective layer praxis (via averaging for homogeneous layers, winner-takes-all for heterogeneous layers). Hybrid layers blend the classical transformation output with the AJN modulation using a configurable alpha parameter. The stimulus object is progressively enriched with features from each layer, and the final output includes the output praxis tensor, cascade risk assessment, saturation status, and a layer-by-layer trace for debugging and monitoring.

# 5. The Artificial Junky Neuron (AJN)

The Artificial Junky Neuron is the foundational building block of PREDATOR. Implemented in src/core/ArtificialJunkyNeuron.js, each AJN is an autonomous computational agent with its own internal state, policy, and life cycle. The AJN model draws explicit inspiration from neuroscience research on addiction: just as a biological addict develops tolerance (requiring more stimulus for the same effect), experiences withdrawal when the stimulus is removed, and may eventually recover (extinction) if the stimulus remains unavailable, an AJN progresses through analogous computational phases.

## 5.1 The Six-Phase Life Cycle

| Phase | Name | Trigger | Behavior |
|---|---|---|---|
| 1 | **RANDOM** | Initialization / post-extinction | High-entropy exploration; praxis sampled from wide Gaussian (sigma=1.0) |
| 2 | **REINFORCE** | Stimulus intensity increasing (delta_alpha > 0) | Policy gradient ascent; mu shifts toward stimulus source; sigma shrinks |
| 3 | **SATURATION** | Stimulus exceeds thetaSat (0.75) | Craving satisfied; praxis suppressed to zero; threshold increases |
| 4 | **WITHDRAWAL** | Threshold decays below craving | Threshold decreases by delta per step; craving returns; output resumes |
| 5 | **FRUSTRATION** | Stimulus intensity not increasing | Covariance sigma expands (chaotic intensification); broader exploration |
| 6 | **EXTINCTION** | Consecutive failures exceed tau (20 steps) | Complete reset: mu=0, sigma=sigmaMax, M=0; revert to RANDOM |

*Table 2: The six-phase AJN life cycle.*

## 5.2 Praxic Policy and Learning

Each AJN maintains a praxic policy parameterized as a Gaussian distribution with per-dimension mean (mu) and standard deviation (sigma), operating in a 64-dimensional praxis space by default. On each processing step, the neuron samples a praxis vector from this distribution using the Box-Muller transform. Learning occurs through two complementary mechanisms. On success (positive stimulus change), the mean shifts toward the gradient direction with learning rate eta (0.05), and the covariance shrinks via exponential decay: sigma(t+1) = sigma(t) * exp(-lambdaSigma * deltaAlpha), where lambdaSigma is 0.10. This narrows the search around successful strategies. On failure (no stimulus improvement), the covariance expands: sigma(t+1) = sigma(t) * exp(gamma * |deltaAlpha|), where gamma is 0.15, increasing exploration in a process the authors term "chaotic intensification." The maximum covariance is capped at sigmaMax (2.0), preventing unbounded exploration.

## 5.3 Default Hyperparameters

| Parameter | Symbol | Default | Description |
|---|---|---|---|
| `betaM` | beta | **0.85** | Exponential smoothing for craving update |
| `lambdaUp` | lambda_up | **0.30** | Saturation ascent rate for threshold |
| `delta` | delta | **0.02** | Metabolic decay rate (withdrawal speed) |
| `thetaSat` | theta_sat | **0.75** | Saturation threshold |
| `tau` | tau | **20** | Extinction horizon (failure steps before reset) |
| `eta` | eta | **0.05** | Praxic learning rate |
| `lambdaSigma` | lambda_sigma | **0.10** | Entropy reduction on success |

| Parameter | Symbol | Default | Description |
|---|---|---|---|
| `gamma` | gamma | **0.15** | Chaotic expansion rate on failure |
| `sigmaMax` | sigma_max | **2.0** | Maximum covariance (extinction reset value) |
| `praximDim` | d | **64** | Dimensionality of praxis tensor |

*Table 3: AJN default hyperparameters.*

# 6. Layer Paradigms and Composition

## 6.1 Paradigm I: Homogeneous AJN Layer

In the homogeneous configuration, all N units in a layer share the same stimulus class and intensity function. The layer output is computed as the average of all unit praxes. A key mechanism is lateral inhibition: each unit perceives a reduced effective stimulus intensity based on the mean craving of its peers, scaled by a coupling coefficient kappa (default 0.2). This prevents all units from simultaneously reaching saturation, which would halt output entirely. The collective saturation threshold rhoSat (default 0.7) defines what fraction of units must be individually saturated before the layer is considered collectively saturated. Homogeneous layers are used for sensory encoding (Layers 1-2, N=64), contextual modulation (Layer 7, N=128), praxic assembly (Layer 11, N=256), and the output emitter (Layer 12, N=1).

## 6.2 Paradigm II: Heterogeneous AJN Layer

The heterogeneous configuration divides the layer into K competing stimulus classes, each with its own set of units and intensity function. The layer processes the stimulus through all K groups in parallel and selects a "winner" class based on which group has the highest mean craving level. This winner-takes-all mechanism enables spontaneous task specialization without explicit supervision: different classes naturally gravitate toward different aspects of the stimulus based on their distinct intensity functions. The implementation includes 8 classes for Layer 3 (syntax, semantics, structure, novelty, relevance, coherence, completion, correctness), 16 classes for Layer 6 (task_progress, context_depth, tool_success, information_gain, plan_adherence, constraint_ok, output_quality, resource_efficiency, owner_alignment, error_absence, exploration_gain, goal_proximity, feedback_richness, action_impact, knowledge_use, completion_signal), and 32 high-order classes for Layer 10 that compute composite signals from lower-level features.

## 6.3 Paradigm III: Hybrid AJN Layer

The hybrid paradigm combines a classical neural transformation with AJN context modulation. The classical function (typically a simulated attention block) processes the stimulus normally, while the AJN sub-layer provides an intrinsic motivation bias. The outputs are blended using a configurable alpha parameter: when the AJN sub-layer is collectively saturated, the modulation is suppressed entirely (intensity set to 0, context="saturated"), otherwise the blended intensity is computed as: intensity = classical.intensity * (1 - alpha) + (praxis_norm / praxis_length) * alpha. Higher alpha

values give more weight to the AJN modulation. The hybrid layers use alpha values that increase with depth (0.3, 0.35, 0.5, 0.6), reflecting the design principle that deeper layers should be more influenced by intrinsic motivation while shallower layers preserve more classical feature extraction.

# 7. Core Modules

## 7.1 Hierarchical Command Interpreter (HCI)

The HCI, implemented in src/modules/HierarchicalCommandInterpreter.js, serves as the bridge between human operators ("Owners") and the neural architecture. It translates natural language directives into structured objects comprising: a goal specification (the raw text), constraint specifications (extracted from patterns like "do not modify X", "prefer reversible", "max N tokens"), resource budgets (token and energy limits scaled by priority), and urgency scalars (routine=0.2, expedited=0.6, critical=0.95). The HCI performs keyword matching to identify which stimulus classes are relevant to the directive, mapping terms like "debug", "fix", "test" to the correctness class, or "search", "research", "find" to the information_gain class. It then builds layer-specific addiction prototypes as Float64Array vectors biased toward the active stimulus classes and injects them into all AJN layers. The HCI also validates proposed praxis actions against Owner constraints, blocking any action that would modify protected files, use unsafe tools, or violate reversible-only requirements.

## 7.2 Token-Energy Arbitrator (TEA)

The TEA, implemented in src/modules/TokenEnergyArbitrator.js, controls the emission rate of the Tensorial Praxic Stream. Its core equation is: r_emit(t) = r0 * exp(-kappaE * E_used/E_budget) * (1 + kappaM * M12(t)), where r0=1.0 is the baseline rate, kappaE=2.0 is the energy suppression coefficient, and kappaM=0.5 is the craving boost coefficient. The TEA tracks token and energy consumption per step, with fixed costs for backbone forward passes (0.001 energy units), variable AJN state updates (0.0005 * craving), and PSE dispatch costs (0.002 * praxis_norm). Token output per step is calibrated as: T_out = floor(praxis_norm * T_out_kappa), where T_out_kappa=10. The critical design insight is that during Saturation phases, the output neuron praxis approaches zero, naturally suppressing token consumption without any external intervention. The authors claim this achieves 3x-5x token efficiency improvements over context-regenerating LLM agents.

## 7.3 Praxic Stream Executor (PSE)

The PSE, co-located in the TEA module file, receives praxis tensors from Layer 12, decodes them into structured tool calls, validates them against HCI constraints, and dispatches them to registered tool handlers (transducers). The decoding process maps praxis tensor values to tool selection indices and extracts priority and argument information. When frustration levels are high (sigma > 1.5), the PSE enters a "chaotic" mode that attempts unconventional tool variations and requires rollback plans. The PSE includes built-in tool stubs for common operations (read_file, write_file, web_search, run_code, api_call, list_dir, memory_store, memory_read, noop) and supports custom

tool registration. All tool executions are recorded in an audit log, providing a complete trace of the agent actions for debugging and accountability.

## 7.4 CascadeMonitor

The CascadeMonitor, implemented in src/modules/CascadeMonitor.js, is a background monitoring system that continuously evaluates the extinction risk across all AJN layers. It defines two thresholds: rhoWarn (default 0.35) for warning-level cascade risk and rhoCritical (default 0.65) for critical-level risk. When cascade risk exceeds the warning threshold, the monitor triggers a stimulus injection, boosting the effective stimulus intensity to rescue starving neurons. When risk exceeds the critical threshold, the monitor escalates to the Owner, pausing the TPS and requesting human intervention. The monitor also logs all extinction events and maintains a history of interventions taken. This three-tier intervention hierarchy (stimulus injection, task decomposition, owner escalation) ensures that the system degrades gracefully rather than collapsing catastrophically when faced with stimulus starvation.

# 8. Training Pipeline

The training pipeline, implemented in src/training/TrainingPipeline.js, follows a four-phase curriculum designed to progressively shape the addiction profiles of all AJN units from random initialization to production-ready behavior. Each phase targets a different aspect of the system capabilities and uses distinct data generation and evaluation strategies.

## 8.1 Phase I: Large-Scale Pre-Training

The first phase (default 10 epochs, batch size 32) runs the backbone forward pass on synthetic stimulus data with random feature values, computing a simulated reconstruction loss. The loss is defined as 0.05 when the output is saturated (indicating the network is correctly suppressing output) and (1 - outputCraving) * 0.5 when unsaturated. This phase initializes the classical transformer blocks and allows AJN units to begin developing their addiction profiles through exposure to diverse stimulus patterns. The data generator produces samples with random intensity values across all stimulus dimensions, providing a broad exploration of the stimulus space.

## 8.2 Phase II: Addiction Shaping

The second phase (15 total epochs across 3 sub-stages of 5 epochs each) uses a curriculum-based approach to progressively shape the addiction dynamics. The Seeding sub-stage provides consistently high-intensity stimuli (0.8-1.0) to reinforce initial addiction formation. The Tolerance sub-stage introduces sinusoidal variation in stimulus intensity, training the neurons to maintain their addiction profiles through moderate stimulus fluctuations. The Frustration sub-stage creates a sparse reward environment where only 30% of samples provide high-intensity stimulus, forcing neurons to develop frustration tolerance and preventing premature extinction. Metrics tracked include saturation rate (fraction of forward passes ending in saturation) and cascade rate (fraction with cascade risk > 0.5).

## 8.3 Phase III: Hierarchical Fine-Tuning (HIFT)

The third phase (8 epochs) introduces Owner directives into the training loop. For each batch sample, the pipeline parses a directive through the HCI, builds layer targets, and injects them into the backbone before running the forward pass. The loss function combines an alignment component (1 - outputCraving * owner_alignment, measuring how well the output craving matches the Owner alignment signal) and a completion component (penalizing unsaturated output when the task should be complete, and vice versa). Default training directives include tasks like "Implement and test the payment processing module" and "Debug and fix the authentication service errors urgently." This phase aligns the intrinsic motivation system with external objectives, bridging the gap between self-organizing behavior and goal-directed action.

## 8.4 Phase IV: Adversarial Frustration Hardening

The fourth and final phase (6 epochs) stress-tests the system with deliberately adversarial stimuli. Most stimulus signals are starved (set to 0.0 or near-zero), with only 30% probability of receiving a high-intensity spike (0.9) and a partial "rescue" signal through novelty (0.0-0.4). After each adversarial step, a recovery step provides rich stimulus to test whether the system can rebound from near-extinction states. Metrics include cascade rate (fraction of adversarial steps triggering cascade risk > 0.4), recovery rate (fraction of recovery steps with outputCraving > 0.3), and resilience (recovery rate / cascade rate). This phase ensures that the trained system can withstand real-world conditions where feedback is sparse, noisy, or adversarial, without collapsing into extinction cascades.

# 9. Hardware Requirements

## 9.1 Current Implementation (Prototype)

The current implementation is a research prototype written in pure JavaScript (Node.js) that performs all computation on the CPU. The architecture uses Float64Array for praxis tensors and standard JavaScript math operations (no GPU acceleration, no tensor framework). The classical transformer blocks are simulated with simple tanh transformations rather than actual matrix multiplications, and the training data is generated synthetically at runtime rather than loaded from disk. As such, the current hardware requirements are minimal.

| Resource | Minimum | Recommended | Notes |
|---|---|---|---|
| **CPU** | Any modern x86/ARM | 4+ cores, 2.0+ GHz | All computation is single-threaded JS |
| **RAM** | 512 MB | 1 GB | Float64Array allocation for ~600 AJN units |
| **Disk** | 50 MB | 100 MB | Node.js + npm dependencies |
| **GPU** | None required | N/A | No GPU support in current implementation |

| Resource | Minimum | Recommended | Notes |
|---|---|---|---|
| **Node.js** | >= 18.0.0 | 20.x LTS | ES modules required |

*Table 4: Hardware requirements for the current prototype.*

## 9.2 Hypothetical Requirements for a Production Implementation

If PREDATOR were to be implemented as a production-grade system with real transformer blocks (replacing the simulated classicalTransformerBlock), actual GPU-accelerated matrix operations, and training on real-world datasets rather than synthetic generators, the hardware requirements would increase substantially. The following analysis is based on the architectural parameters defined in the codebase and standard deep learning scaling assumptions.

The total number of AJN units across all layers can be estimated as follows: Layer 1 has 64 units, Layer 2 has 64 units, Layer 3 has 8x8=64 units, Layer 6 has 16x8=128 units, Layer 7 has 128 units, Layer 10 has 32x4=128 units, Layer 11 has 256 units, and Layer 12 has 1 unit, totaling approximately 833 AJN units. Each unit maintains a 64-dimensional praxis space (mu and sigma arrays), plus several scalar state variables. The total parameter count for the AJN layers alone is approximately 833 * (64 + 64 + 5) = 110,789 parameters. However, the classical transformer blocks (Layers 4-5, 8-9) and the convolution/fully-connected components of the hybrid layers would add significantly more parameters if implemented properly. A conservative estimate for a full implementation with standard transformer dimensions would be 10M-100M total parameters, comparable to a small-to-medium language model.

| Resource | Training | Inference |
|---|---|---|
| **GPU** | 1-4x NVIDIA A100 (40/80 GB) or equivalent | 1x NVIDIA T4 or RTX 3090 (24 GB) |
| **RAM** | 64-256 GB | 16-32 GB |
| **CPU** | 16+ cores, 3.0+ GHz | 8 cores, 2.5+ GHz |
| **Disk** | 500 GB SSD (training data + checkpoints) | 50 GB SSD (model weights + tools) |
| **Training Time (estimated)** | 2-14 days (4 phases, ~40 epochs total) | N/A |
| **Inference Latency (estimated)** | N/A | 50-200ms per step (200 steps/task max) |

*Table 5: Estimated hardware requirements for a production implementation.*

The key differentiator for PREDATOR inference costs is the TEA mechanism. During Saturation phases, the output praxis approaches zero, meaning the system naturally suppresses token generation when it perceives the task is progressing well. This could result in significantly lower per-task inference costs compared to traditional LLM agents that regenerate their full context window on every step. However, during Frustration phases, the expanded covariance leads to higher praxis norms and thus higher token output, meaning costs spike precisely when the agent is struggling, which is the opposite of the desired economic behavior. The overall efficiency therefore

depends heavily on the ratio of Saturation to Frustration steps in production workloads, which remains an open empirical question.

# 10. Implementation Assessment

## 10.1 Code Quality and Architecture

The codebase demonstrates strong architectural discipline. Each module has a clear single responsibility, well-documented API surfaces, and comprehensive JSDoc comments that reference the specific equations and sections from the Agentic Theory preprints. The event-driven architecture (using EventEmitter3) enables loose coupling between components, making the system observable and extensible. The separation of concerns between the neural backbone (ANNPsi), command interpretation (HCI), resource management (TEA), action execution (PSE), and safety monitoring (CascadeMonitor) is clean and follows established software engineering patterns. The use of ES modules, modern JavaScript features (optional chaining, nullish coalescing), and a well-structured package.json with appropriate scripts for development, testing, and deployment indicates professional-grade project scaffolding.

## 10.2 Limitations and Gaps

The most significant limitation is that the current implementation uses simulated components for the classical neural network layers. The classicalTransformerBlock function in ANNPsi.js applies simple tanh transformations with random noise rather than performing actual matrix multiplications with learned weight matrices. This means the "transformer" layers do not implement real self-attention, and the system as a whole cannot learn meaningful feature representations from data. Similarly, the training pipeline uses synthetic data generators that produce random stimulus values rather than loading real-world training data. The loss functions are heuristic approximations rather than derivable from a formal optimization objective. These design choices are understandable for a research prototype that aims to validate the architectural concept, but they mean the current implementation cannot be trained to perform any real-world task.

Additional gaps include the absence of: (1) a persistence mechanism for saving and loading trained model weights; (2) a real LLM integration layer that would enable the agent to process actual natural language inputs; (3) gradient computation and backpropagation for the classical layers, which would be necessary for end-to-end training; (4) multi-agent coordination protocols that the Agentic Theory papers suggest as future work; and (5) benchmarking against baseline LLM agents on standard agentic tasks. The README explicitly acknowledges the training requirement with a prominent warning, suggesting the author intends to address these gaps in future iterations.

## 10.3 Novel Contributions

Despite the prototype limitations, PREDATOR introduces several genuinely novel concepts to the AI agent landscape. The AJN life-cycle model, with its addiction-inspired dynamics of craving, tolerance, withdrawal, and extinction, provides a biologically-grounded alternative to the standard

reinforce/punish paradigm of reinforcement learning. The TEA emission rate equation, which naturally suppresses output during satisfaction and amplifies it during craving, offers an elegant solution to the token efficiency problem in LLM agents. The CascadeMonitor intervention hierarchy, with its graduated response from stimulus injection to owner escalation, addresses the critical reliability problem of autonomous agents. The HCI mechanism for projecting natural language directives into distributed addiction targets across a deep hierarchy is an innovative approach to alignment that avoids the fragility of prompt engineering. And the four-phase training curriculum, particularly the adversarial frustration hardening phase, demonstrates awareness that production agents must be resilient to sparse and adversarial feedback, not just optimized for average-case performance.

# 11. Conclusions

PREDATOR represents a bold and conceptually rich departure from the dominant paradigm of LLM-based AI agents. Its core thesis, that autonomous agents should be driven by intrinsic motivation modeled after addiction dynamics rather than external reward signals, is both scientifically grounded (drawing on neuroscience literature) and practically motivated (addressing real limitations of current agent architectures). The 12-layer ANN-Psi backbone, with its three compositional paradigms and six-phase neuronal life cycle, provides a flexible and theoretically coherent framework for building agents that can self-organize, self-regulate, and gracefully degrade under adversarial conditions.

The current implementation serves as an architectural proof-of-concept rather than a production-ready system. The simulated classical layers, synthetic training data, and approximate loss functions limit its immediate practical applicability. However, the codebase provides a well-structured foundation that could be extended with real neural network components (using frameworks like PyTorch or TensorFlow.js), actual LLM integration for natural language understanding, and real-world training datasets. The theoretical framework is sufficiently detailed to guide such extensions, with formal equations for every component and explicit references to the underlying preprints.

In terms of hardware requirements, the current prototype can run on any modern computer with Node.js 18+ and minimal resources (under 1 GB RAM, no GPU). A production implementation with real transformer blocks and proper training would likely require GPU clusters comparable to those used for medium-sized language models (1-4x NVIDIA A100 for training, 1x consumer GPU for inference), with training times estimated at 2-14 days depending on dataset size and model dimensions. The claimed 3x-5x token efficiency improvement over LLM agents, if validated empirically, would represent a significant practical advantage, though this remains to be demonstrated in real-world benchmarks. The PREDATOR project, despite its early stage, contributes valuable ideas to the agentic AI research community and merits continued development and evaluation.